

RECLAIM Project

DIGT 2107 - Project Iteration 3.2: Completed Implementation

RECLAIM[Home](#)[Wellness ▼](#)[Support ▼](#)[Activities ▼](#)[Login](#)[Sign Up](#)

Reclaim Your Wellness

Track your mood, build healthy habits, and access support resources designed for university students.

How are you feeling today?

Track your daily mood to understand your wellness patterns



Great



Good



Okay



Not Good



Terrible

Add a note (optional)

What's on your mind today?

Save Mood Entry

Recent Mood Entries

Please [log in](#) to track your mood.

1

1 INTRODUCTION	3
2 SUMMARY OF REFACTORING ACTIONS AND CODEBASE IMPROVEMENTS	4
3 RESULTS OF TESTING AND VALIDATION	7

1. INTRODUCTION

Course: DIGT2107 – Fall Term 2025

Instructor: Dr. May Haidar

Due Date: March 8, 2026

Project name: Reclaim

Team number: Group 4

Team members: Brianna Nguyen, Justin Gamez, Kieran Bendell, Ruqayya Naqvi

GitHub repository: <https://github.com/justingmz77/Reclaim>

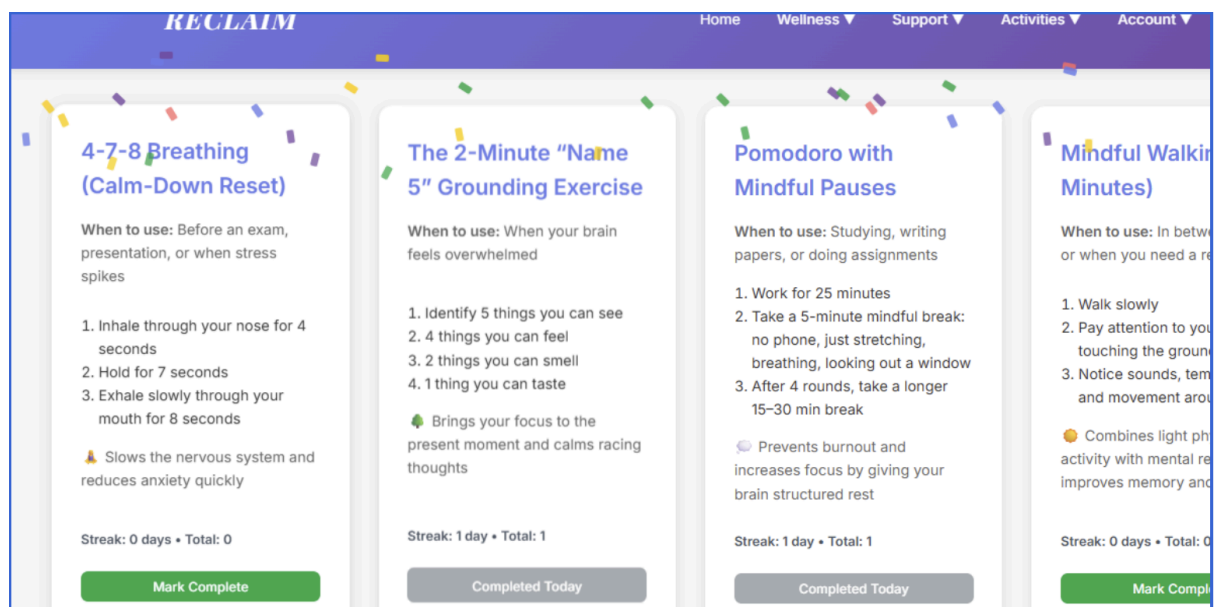
2. SUMMARY OF REFACTORING ACTIONS AND CODEBASE IMPROVEMENTS

1) Fixing a small bug on the “Wellness Tools” page.

If a user clicked “Completed”, the button would be consistently greyed out on every new log in, session, and day, effectively not allowing the user to track their completion.

To fix this, I changed public/wellnessTools.js from a one-time global completion list (completedTools) to a per-user daily progress model stored in localStorage with userDataManager.getUserStorageKey(...). Now, clicking “Mark Complete” only locks it for today, updates streak logic (continue if yesterday was completed, otherwise reset), and auto-resets the button the next day. I also added a small confetti effect on completion and a streak/total label per card.

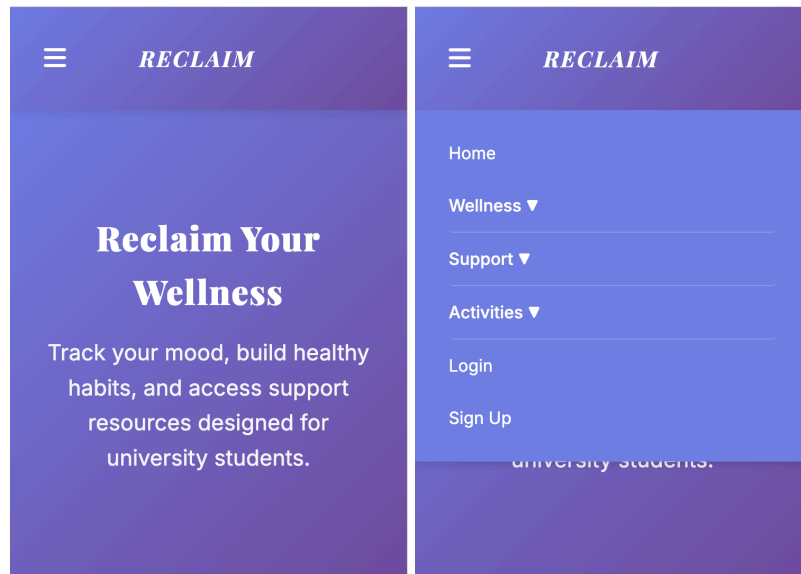
This was a needed improvement to ensure a good user experience.



2) More responsive navigation bar for tablet and mobile

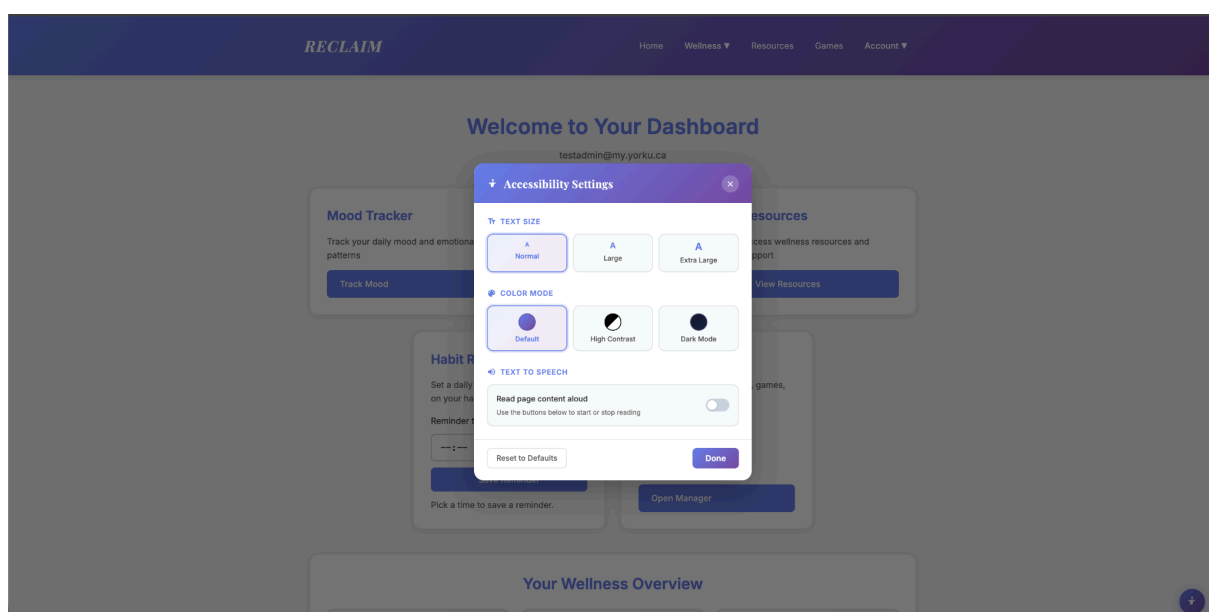
Between the viewports 785px and 768px, the navigation bar would not nicely display the items. For example, not all navigation items would display on the same line. Additionally, on mobile, all the navigation items would always display in a dropdown. This UI was distracting because it took up a large portion of the screen with no ability to collapse the navigation bar.

Now, when the viewport is 785px or less, a hamburger icon appears that allows the user to open or collapse the navigation bar. This enables a more responsive, smoother UI and user experience for mobile/tablet users.



3) Accessibility Settings

We added an Accessibility Settings panel to the app so that more people can use it comfortably. You can open it anytime by clicking the small wheelchair icon button in the bottom-right corner of the screen, or by going to the Account menu in the navigation bar and clicking "Accessibility." Once it's open, you'll see three options: you can change the text size to Normal, Large, or Extra Large, you can switch the color mode between Default, High Contrast, or Dark Mode, and you can turn on Text-to-Speech so the page can be read out loud to you. Whatever settings you choose will be saved automatically. The next time you visit the app, even on a different page, your preferences will still be there. For example, if you turn on Dark Mode and set the text to Large, those changes will stay until you reset them. The goal is to make the app easier and more comfortable to use for everyone, no matter how they like to read or view things on screen.



4) Refactoring the navigation rendering and simplifying the logic

This refactor simplifies the shared navigation script by removing unnecessary complexity and improving maintainability. The `isFlat` property was removed from navigation categories because single-item links like Login and Sign Up can be handled the same way as Home by checking the number of items, eliminating extra conditional logic. HTML attribute construction for links was streamlined by building attributes declaratively with array expressions and `.filter(Boolean)`, removing repetitive push operations and making the code easier to read. The admin navigation item is now inserted inline when defining the category using the spread operator, rather than mutating the array afterward, which makes the structure more predictable and easier to maintain. Finally, the logout function was simplified by consolidating redirects into a `finally` block, ensuring consistent behavior and removing duplicate code. Overall, these changes reduce special-case handling, improve readability, and make the navigation system more robust and easier to extend.

5) Add aria labels to make the content more accessible for screen readers

We added aria labels for the hamburger icon that opens and collapses the navigation bar for tablet and mobile devices. This was a necessary change because it provides context about the element to screen readers when it may not be obvious. In addition, we used aria labels to hide certain elements from being read. For example, we hid the hamburger icon because it is an icon and emojis because sometimes they are weirdly read by the screen readers.

6) Fixed journal entry bug

While testing the journal entry feature with the mood tracker, we found a critical bug in the journal feature where entering a mood could accidentally populate and save the journal form, causing incorrect journal entries. The changes included updating the HTML to use distinct IDs for journal title and content fields, separating them from mood tracking fields, and revising the JavaScript logic to correctly handle journal entry submission, refresh the journal history, and display success messages. These changes were necessary to ensure that journal entries are saved accurately and independently of mood inputs, preventing data corruption and user confusion. The impact on the overall system is that the journal functionality now works reliably, maintaining proper separation between mood tracking and journal entries, improving data integrity and user experience.

7) Refactoring style sheet to reflect general class names for journal and mood

I refactored the shared mood/journal UI so both pages now use a single generic tracker class instead of the previous naming which was prefixed with “mood”. This was an important change because before both journal and mood were using these styling classes which is not correct convention and can make the code confusing.

3. RESULTS OF TESTING AND VALIDATION

To validate the changes made during this iteration, we built and ran a full test suite covering three layers of the application: unit tests, integration tests, and end-to-end tests. All 384 tests passed across 14 test suites, with an overall code coverage of 83% for statements and lines, 86.6% for branches, and 95.9% for functions.

File	% Stmts	% Branch	% Funcs	% Lines
All files	83.04	86.56	95.89	82.94
Reclaim	82.56	84.58	92.5	82.38
server.js	82.56	84.58	92.5	82.38
Reclaim/utils	100	100	100	100
auth.js	100	100	100	100
Reclaim/utils/db	82.09	96.96	100	82.09
GameScores.js	76.92	100	100	76.92
Habits.js	80.32	100	100	80.32
JournalEntries.js	75.75	100	100	75.75
MoodEntries.js	75.75	100	100	75.75
Users.js	86.53	100	100	86.53
db.js	100	75	100	100
index.js	100	100	100	100
Test Suites: 14 passed, 14 total				
Tests: 384 passed, 384 total				
Snapshots: 0 total				
Time: 34.836 s, estimated 46 s				
Ran all test suites.				

Unit Tests

Unit tests were written to verify the core logic of the application in isolation. This included the authentication module (email validation, password rules, YorkU domain enforcement) and all five database modules: Users, MoodEntries, JournalEntries, GameScores, and Habits. Each module was tested for its full CRUD lifecycle, adding, retrieving, updating, and deleting records, as well as edge cases like duplicate entries, invalid inputs, missing users, and foreign key constraints. The database schema itself was also validated to confirm that all tables, columns, and indexes were created correctly. Authentication logic reached 100% coverage. The database utilities averaged 82% line coverage.

Integration Tests

Integration tests were written to verify that the server routes behave correctly end-to-end, from HTTP request to database response. This covered the authentication routes (signup, login, logout), mood and journal entry routes, game score submission and leaderboard retrieval, and the full admin API including user management (list, create, update role, delete). Each route was tested for both happy-path and error-path scenarios, for example, confirming that a student receives a 403 when attempting to access admin-only routes, and that unauthenticated requests are rejected with a 401. The server reached 82.6% statement coverage, with the uncovered lines primarily being error branches that are only reachable under extreme database failure conditions.

End-to-End Tests

End-to-end tests were written using Playwright to simulate real user interactions in a browser. These tests covered the full user journey across the major features of the app: signing up with a YorkU email, logging in, tracking a mood entry, writing a journal entry, using the wellness tools, playing games, and navigating the admin panel. Admin content management was tested thoroughly, creating, editing, and deleting journaling prompts, exercises, and game entries, as well as access control, confirming that students are blocked from admin pages and that content changes made through the admin panel are immediately reflected in the public API. A total of 7 E2E spec files were written, targeting the features most directly affected by the changes in this iteration.